SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF SCIENCE AND HUMANITIES

DEPARTMENT OF COMPUTER APPLICATIONS

SRM

INSTITUTE OF SCIENCE & TECHNOLOGY

Deemed to be **University** *u/s 3 of UGC Act, 1956*

PRACTICAL RECORD NOTE

STUDENT NAME :

REGISTER NUMBER :

CLASS : MCA SECTION : E

YEAR & SEMESTER : I YEAR & II SEMESTER

SUBJECT CODE : PCA25S02J

SUBJECT TITLE : INTERNET OF THINGS (IOT)

APRIL 2026

SRM

INSTITUTE OF SCIENCE & TECHNOLOGY

Deemed to be **University** *u/s 3 of UGC Act, 1956*

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF SCIENCE AND HUMANITIES

DEPARTMENT OF COMPUTER APPLICATIONS

SRM Nagar, Kattankulathur – 603 203

CERTIFICATE

Certified to be the bonafide record of practical work done by

---

Register No. _____ *of* **MCA** *Degree programme for* **PCA25S02J –** ***INTERNET OF THINGS (IOT)*** *in the Computer lab in SRM Institute of Science and Technology during the academic year 2025- 2026.*

Staff In-charge *Head of the Department*

Submitted for Semester Practical Examination held on _____.

Internal Examiner *External Examiner*

INDEX

EX NO: 1 Date: 4.12.2025	Arduino Installation and LED ON/OFF using Node MCU Esp8266

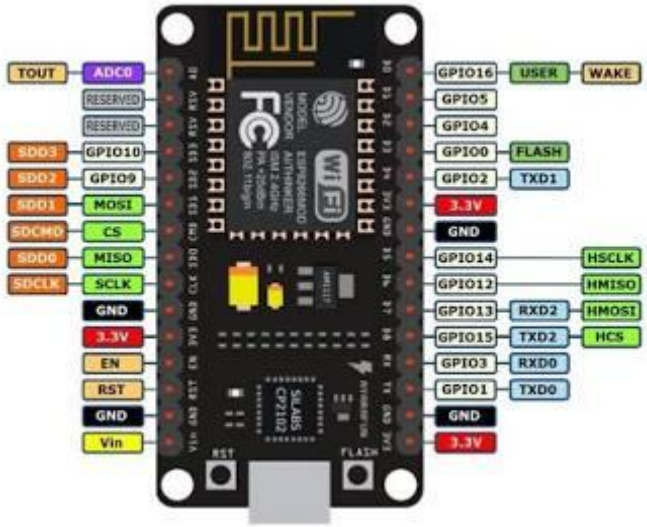
AIM:

To install and set up the Arduino IDE for NodeMCU (ESP8266) and to interface an LED with the NodeMCU to control its ON and OFF operation through programming.

Components

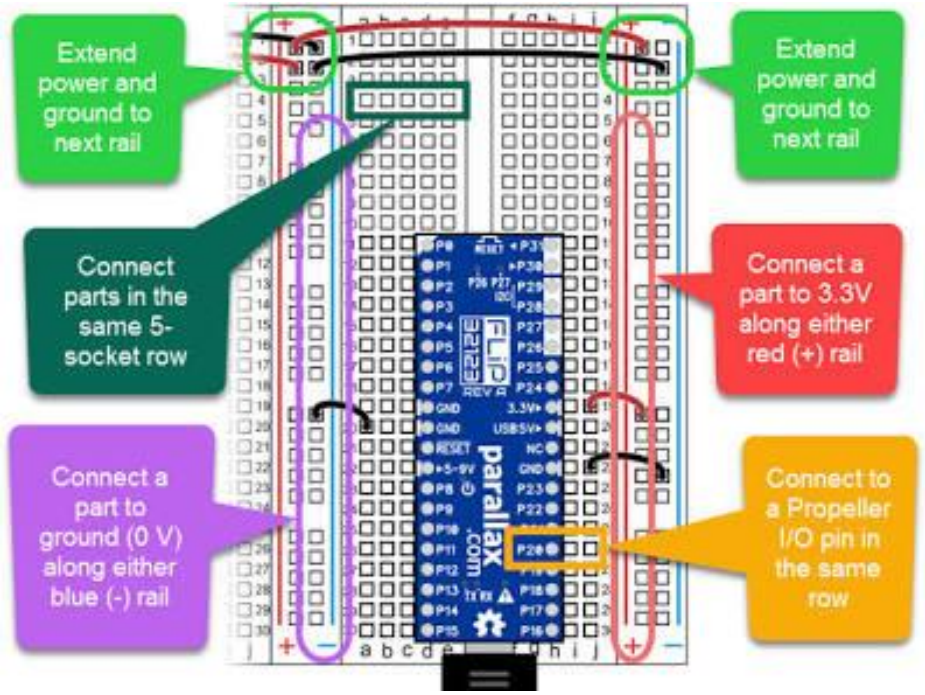
- Here are the hardware components you will need for this lab:
- NodeMCU ESP8266 Development Board
- Description: This is a low-cost, open-source IoT platform that includes the ESP-12E Wi-Fi module, a built-in programmer, and breakout pins. It allows you to program the ESP8266 Wi-Fi microcontroller using the Arduino IDE. h

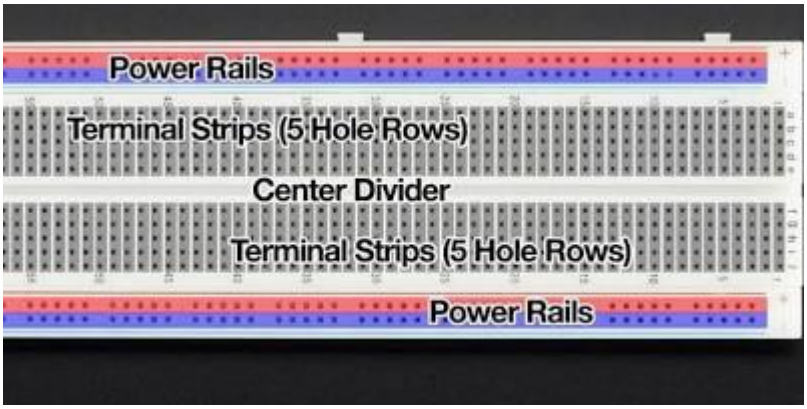
NodeMCU ESP8266 PINOUT.



Breadboard

Description: A solderless board used for prototyping electronic circuits. It has rows of holes connected internally by metal strips. The long outer columns (power rails) are connected vertically, and the inner rows are connected horizontally in sets of five holes. This lets you easily plug in components and jumper wires without soldering





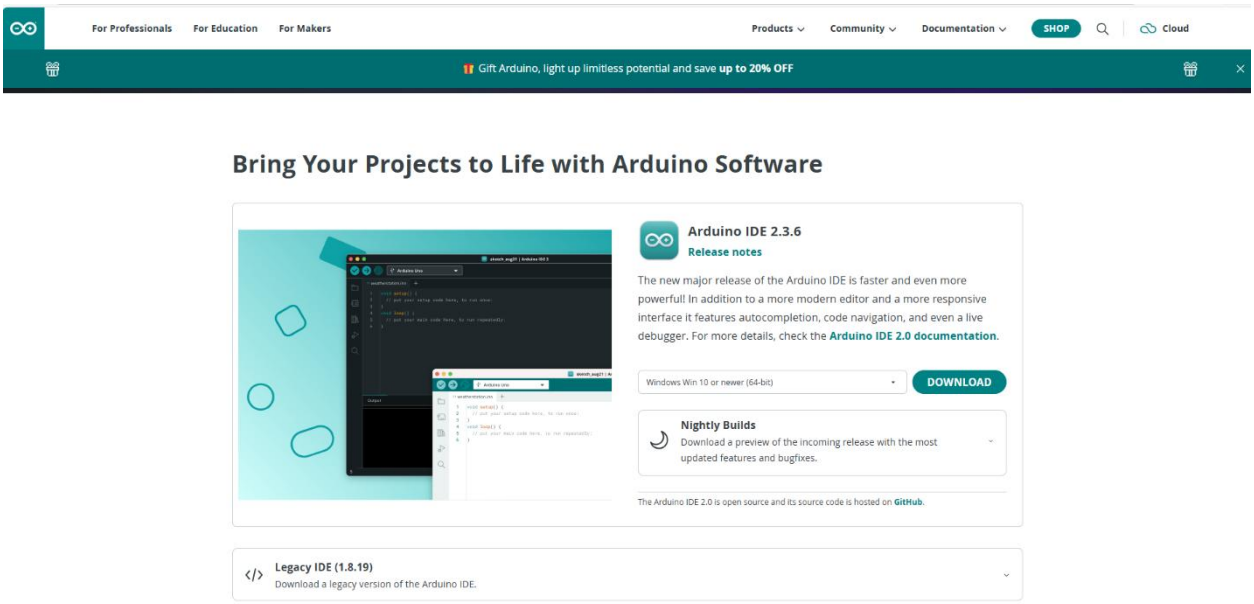
LED (Light Emitting Diode)

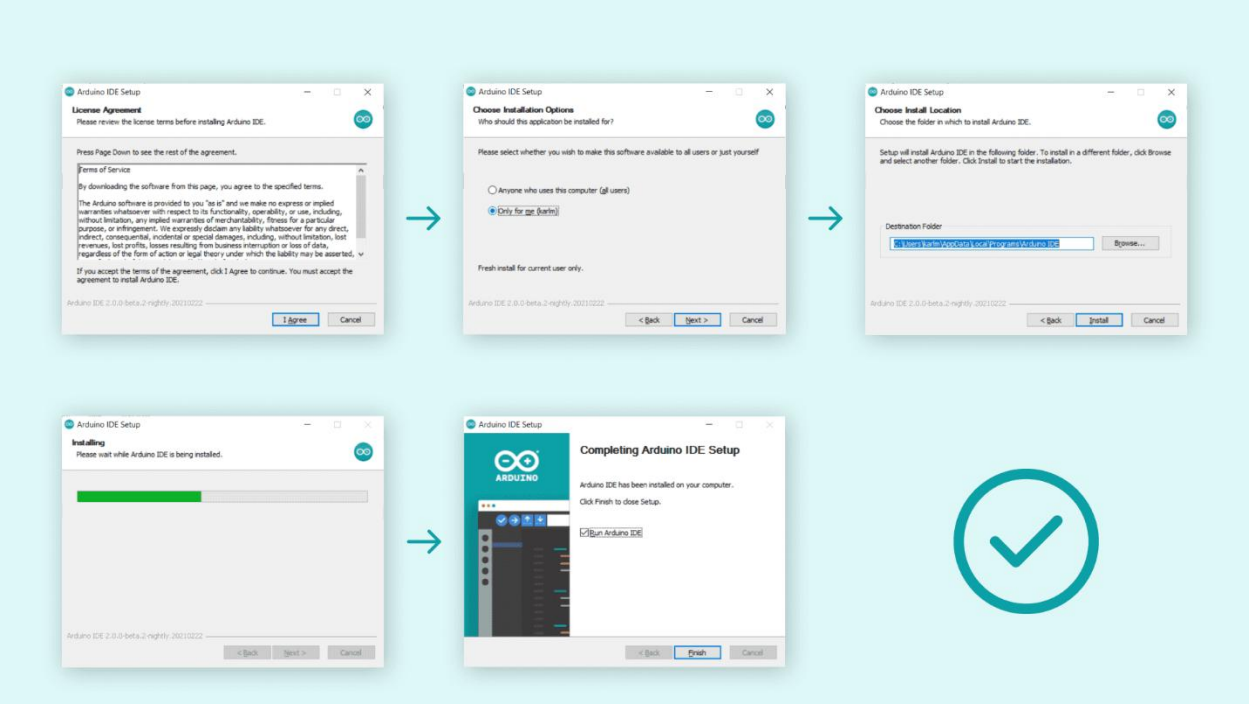
Description: A two-legged electronic component that produces light when current flows through it in the correct direction.

It has two legs: the anode (longer leg), which connects to the positive side (power/GPIO pin), and the cathode (shorter leg), which connects to the negative side (ground/GND).

Arduino Installation and Blink LED using Node MCU Esp8266

Arduino Installation





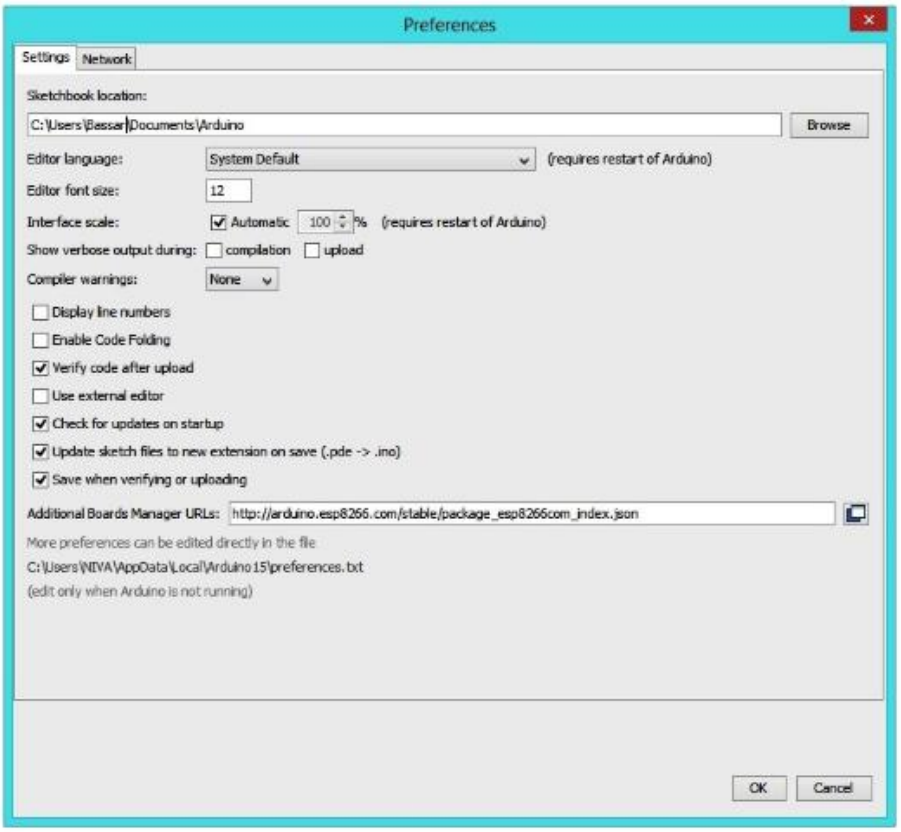
Part 1: Arduino IDE Setup for NodeMCU

Add ESP8266 Boards URL:

Open Arduino IDE > File > Preferences (or Arduino IDE > Settings on Mac).

Installing the NodeMCU Support for the Arduino

- First open the Arduino IDE
- Go to files and click on the preference in the Arduino IDE

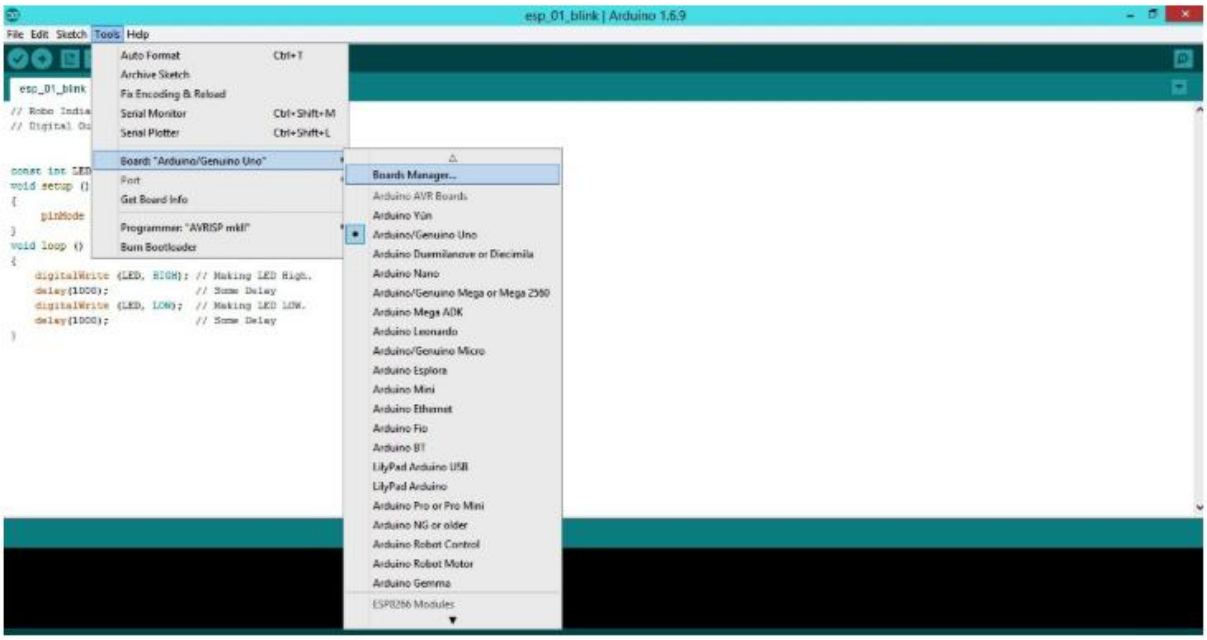


- Paste below URL in the Additional boards Manager

In "Additional Boards Manager URLs," paste: http://arduino.esp8266.com/stable/package_esp8266com_index.json. Click OK.

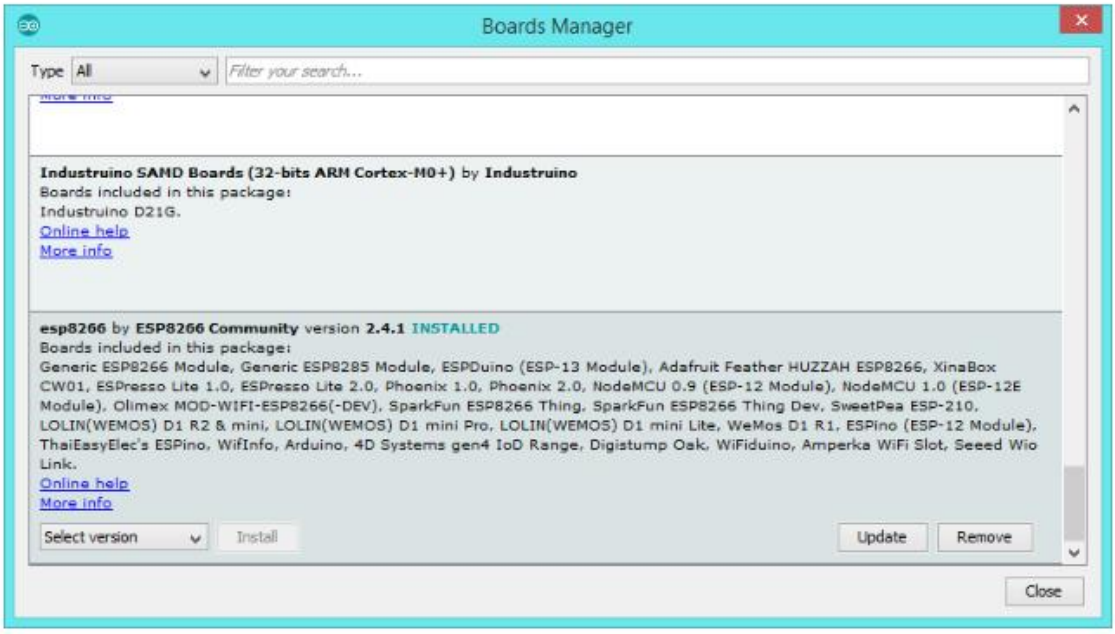
Install ESP8266 Package:

Go to Tools > Board > Boards Manager.



Search for esp8266 and install the "esp8266 by ESP8266 Community" package.

- Navigate to ESP8266 by ESP8266 community and install the software.

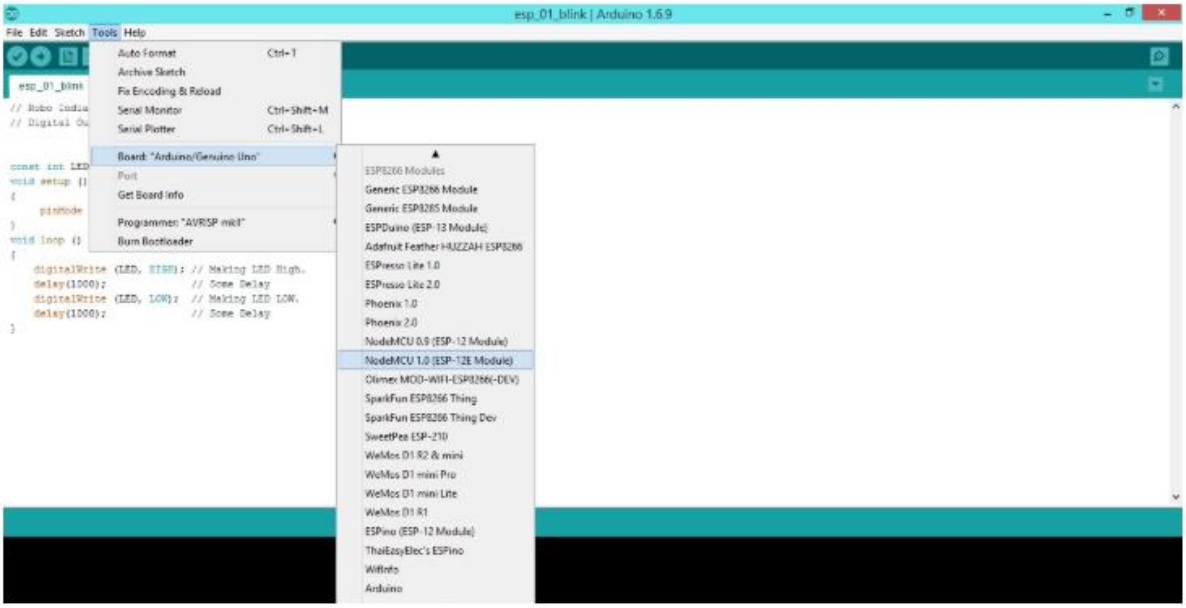


- Once all the above process is completed we are ready to program our NodeMCU with Arduino IDE.

Connect NodeMCU: Plug your NodeMCU board into your computer with a micro-USB cable. Check Device Manager (Windows) or System Information (Mac) for the assigned COM port (e.g., COM3).

Select Board & Port:

In Tools > Board, select "NodeMCU 1.0 (ESP-12E Module)" or your specific model.



- Then select the correct COM port to upload the program on your NodeMCU
- Now Copy Paste the below programme and upload it.
- If you did everything correctly then, your led should blink continuously.

In Tools > Port, select the COM port detected in the previous step.

Go to Tools → Port → COM3 (If COM3 is unavailable download CP210x Windows Drivers from the link https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers?tab=downloads)

Code

Blink Inbuilt LED of NODEMCU c_cpp

Example code for Blinking Inbuilt LED of NODEMCU

```
void setup() {
  // initialize inbuilt LED pin as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// loop function runs over and over  again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the  LED on by making the pin 13 HIGH
  delay(500);              // wait for a 0.5  second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the  pin 13 LOW
  delay(500);              // wait for a 0.5 second
}
```

Part 2: Code

// Pin connected to LED (NodeMCU built-in LED is often D0 or D4)

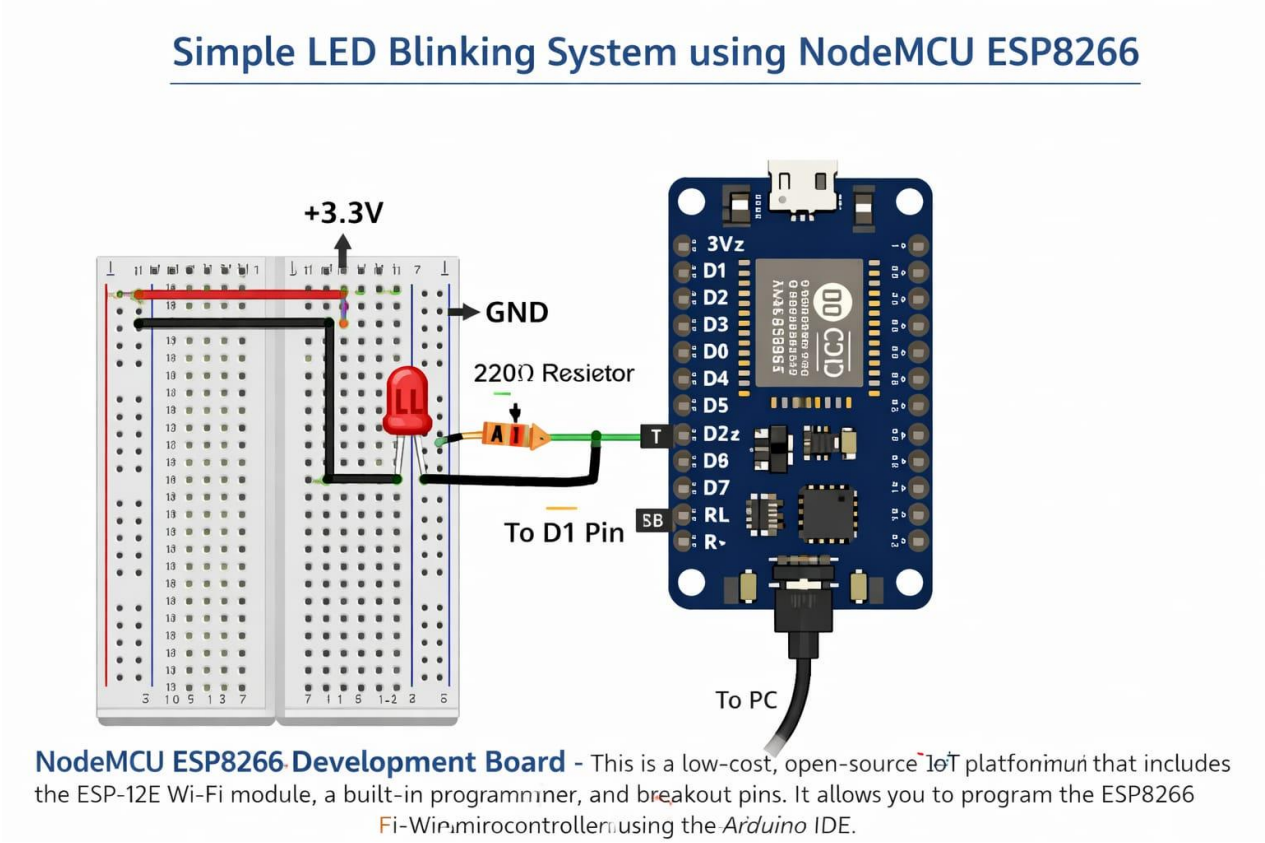
```
#define LED_PIN D5 // Or D0, check your board's pinout


void setup() {
  pinMode(LED_PIN, OUTPUT); // Initialize digital pin as an output.
}
void loop() {
  digitalWrite(LED_PIN, LOW);
  digitalWrite(LED_PIN, HIGH);
}
```

Part 3: Upload and Verify

Upload Sketch: Click the Upload button (right arrow icon) in the Arduino IDE

Observe: The built-in LED on the NodeMCU should start blinking (on for 1 second, off for 1 second).

Output:



RESULT:

The program has been executed successfully.

EX NO: 2	BLINK LED using Node MCU
Date: 4.12.2025	

AIM:

To interface an LED with NodeMCU (ESP8266) and program it to blink at regular intervals using Arduino IDE.

Components Required:

- NodeMCU ESP8266
- LED
- Breadboard
- Jumper wires
- USB cable
- Arduino IDE

Connections:

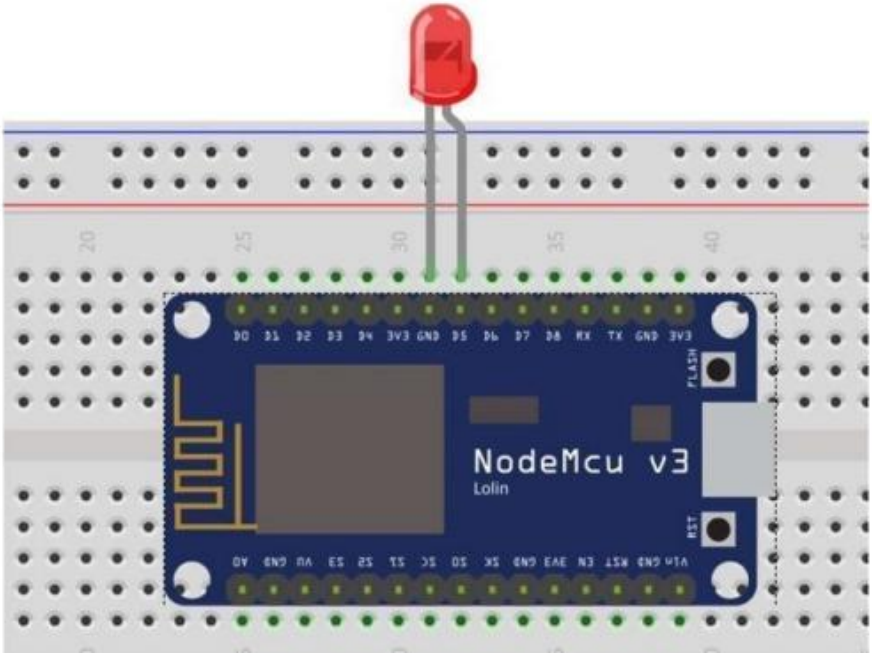
LED → D5

GND → GND

Program:

```
#define LED_PIN D5

void setup() {
 pinMode(LED_PIN, OUTPUT); // Initialize digital pin as an output.
 }


 void loop() {
   digitalWrite(LED_PIN, LOW);
   delay(1000);              // Wait for a second.
   digitalWrite(LED_PIN, HIGH);
   delay(1000);              // Wait for a second.
 }
```

Output:



RESULT:

The program has been executed successful

EX NO: 3	Quiz Program using NodeMCU ESP8266
Date: 11.12.2025	

AIM:

To develop an interactive quiz system using NodeMCU ESP8266 and Serial Monitor with LED indication.

Objective:

- Learn Serial communication
- Accept user input from Serial Monitor
- Implement decision making using conditions
- Control LED based on answer

Components Required:

- NodeMCU ESP8266
- LED
- Breadboard
- Jumper wires
- USB cable
- Arduino IDE

Theory:

NodeMCU communicates with a computer using Serial communication. A quiz question is displayed in Serial Monitor. The user enters an answer, and NodeMCU checks correctness to control an LED.

Connections:

LED → D5

GND → GND

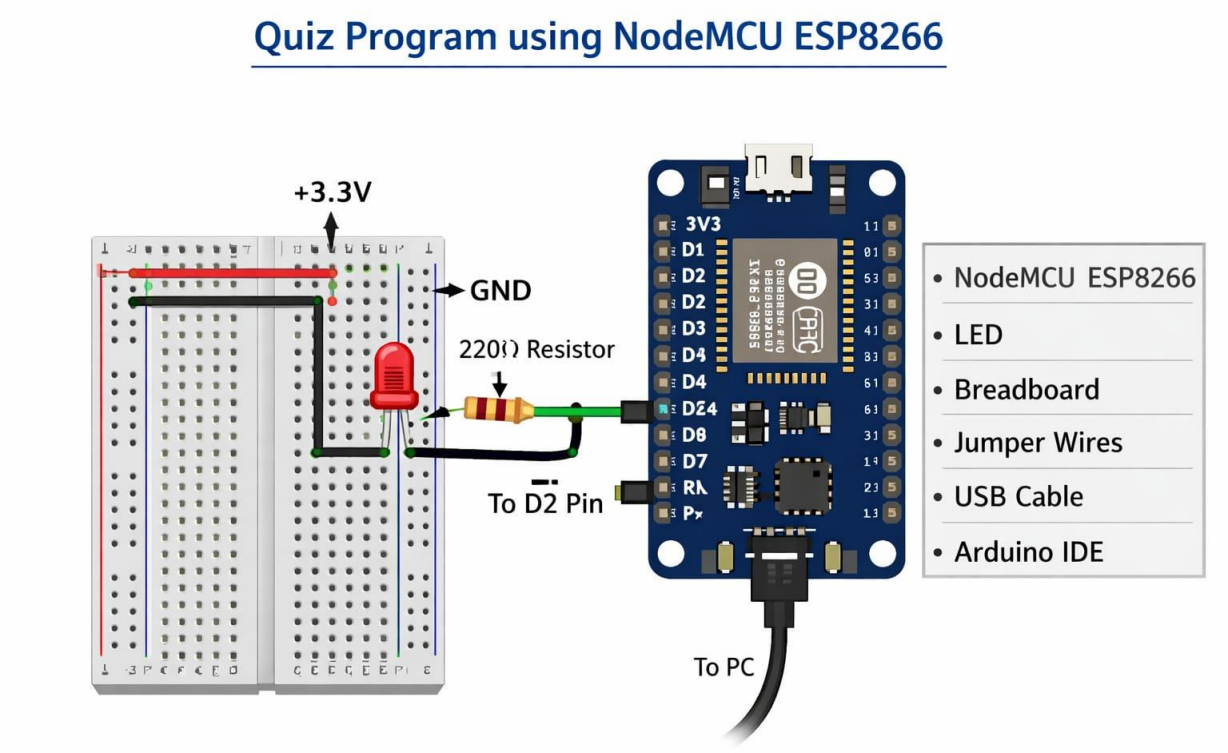
Program:

```
String userAnswer = "";
const int LED_PIN = D5;

void setup() {
  Serial.begin(9600);
  delay(1000);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  Serial.println("=== QUIZ STARTED ===");
  Serial.println("Question:");
  Serial.println("What is the capital of India?");
  Serial.println("Type your answer and press Enter:");
}

void loop() {
  if (Serial.available() > 0) {
    userAnswer = Serial.readStringUntil('
');
    userAnswer.trim();
    userAnswer.toLowerCase();

    if (userAnswer == "delhi" || userAnswer == "new delhi") {
      Serial.println("Correct Answer! LED ON");
      digitalWrite(LED_PIN, HIGH);
    } else {
      Serial.println("Wrong Answer! Try again");
      digitalWrite(LED_PIN, LOW);
    }

    Serial.println();
    Serial.println("Type your answer again:");

   }
 }
```

Procedure

1. Connect LED to NodeMCU
2. Upload the program
3. Open Serial Monitor
4. Enter the answer
5. Observe LED behavior

Output:



RESULT:

The program has been executed successful

EX NO: 4	IR Sensor with NodeMCU ESP8266
Date: 18.12.2025	

AIM:

To interface an Infrared (IR) obstacle sensor with NodeMCU ESP8266 and detect objects using LED indication.

Objective:

- Understand IR sensor working
- Learn digital input interfacing
- Control LED using NodeMCU
- Display output in Serial Monitor

Components Required:

- NodeMCU ESP8266
- IR obstacle sensor
- LED
- Breadboard
- Jumper wires
- USB cable
- Arduino IDE

Theory:

An IR sensor works using infrared light. It emits IR rays and detects reflections from nearby objects. When an object is detected, the output becomes LOW and can be read by NodeMCU to trigger actions.

Connections:

IR OUT → D6
IR VCC → 3.3V
IR GND → GND
LED → D5

Program:

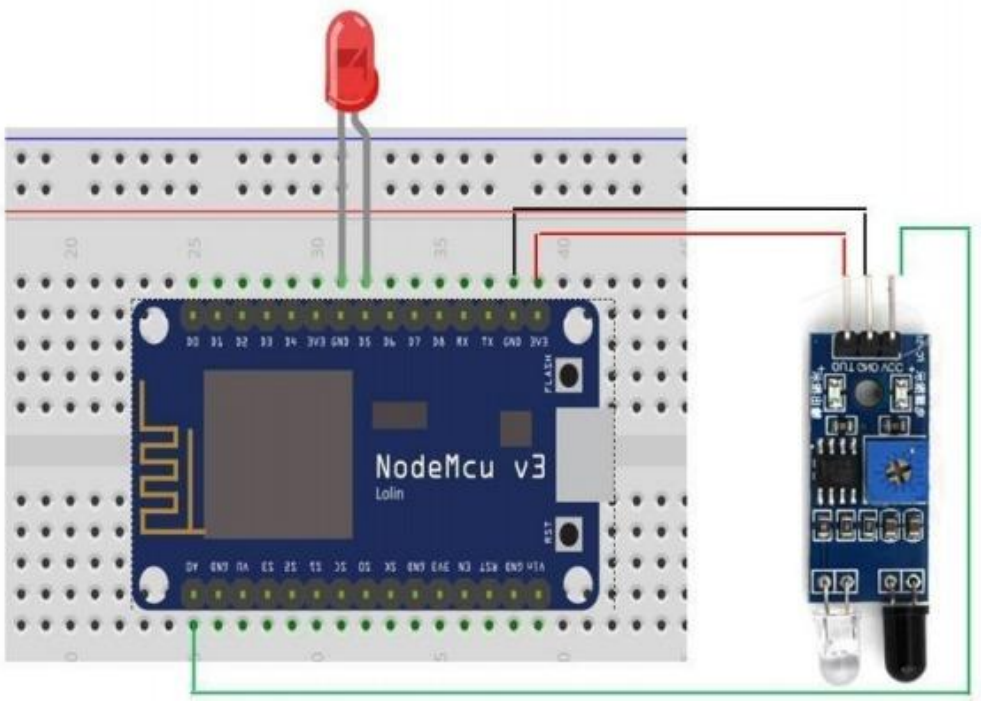
```
const int IR_PIN = D6;
const int LED_PIN = D5;

void setup() {
  Serial.begin(9600);
  pinMode(IR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
}
```

```
void loop() {
  int sensorState = digitalRead(IR_PIN);

  if (sensorState == LOW) {
    Serial.println("Object Detected");
    digitalWrite(LED_PIN, HIGH);
  } else {
    Serial.println("No Object");
    digitalWrite(LED_PIN, LOW);
  }

  delay(300);
}
```

Procedure:

1. Connect the circuit
2. Upload the program
3. Open Serial Monitor
4. Move object near sensor
5. Observe LED and output

Output:



RESULT:

The program has been executed successfully

EX NO: 5	Clap Controlled LED using Sound Sensor
Date: 06.01.2026	

AIM:

To design a system using a **sound sensor and an Arduino** where:

- **1 Clap → LED bulb ON**
- **2 Claps → LED bulb OFF**

.

Objective:

- To detect sound using a sound sensor.
- To control an LED bulb using clap signals.
- To demonstrate a simple **sound-controlled automation system**.

Components Required:

- Arduino Uno
- Sound Sensor Module KY-038
- LED bulb / LED
- 220Ω Resistor
- Breadboard
- Jumper wires
- USB cable

Circuit Connections:

Component	Arduino Pin
Sound Sensor DO	Pin 2
Sound Sensor VCC	5V
Sound Sensor GND	GND
LED Positive	Pin 13
LED Negative	GND (through resistor)

.

Program:

```
int soundSensor = 2;
int led = 13;
int clapCount = 0;

void setup() {
  pinMode(soundSensor, INPUT);
```

```
  pinMode(led, OUTPUT);
}

void loop() {

  if (digitalRead(soundSensor) == HIGH) {
    clapCount++;
    delay(500);
  }

  if (clapCount == 1) {
    digitalWrite(led, HIGH);   // LED ON
  }

  if (clapCount == 2) {
    digitalWrite(led, LOW);    // LED OFF
    clapCount = 0;
  }
}
```

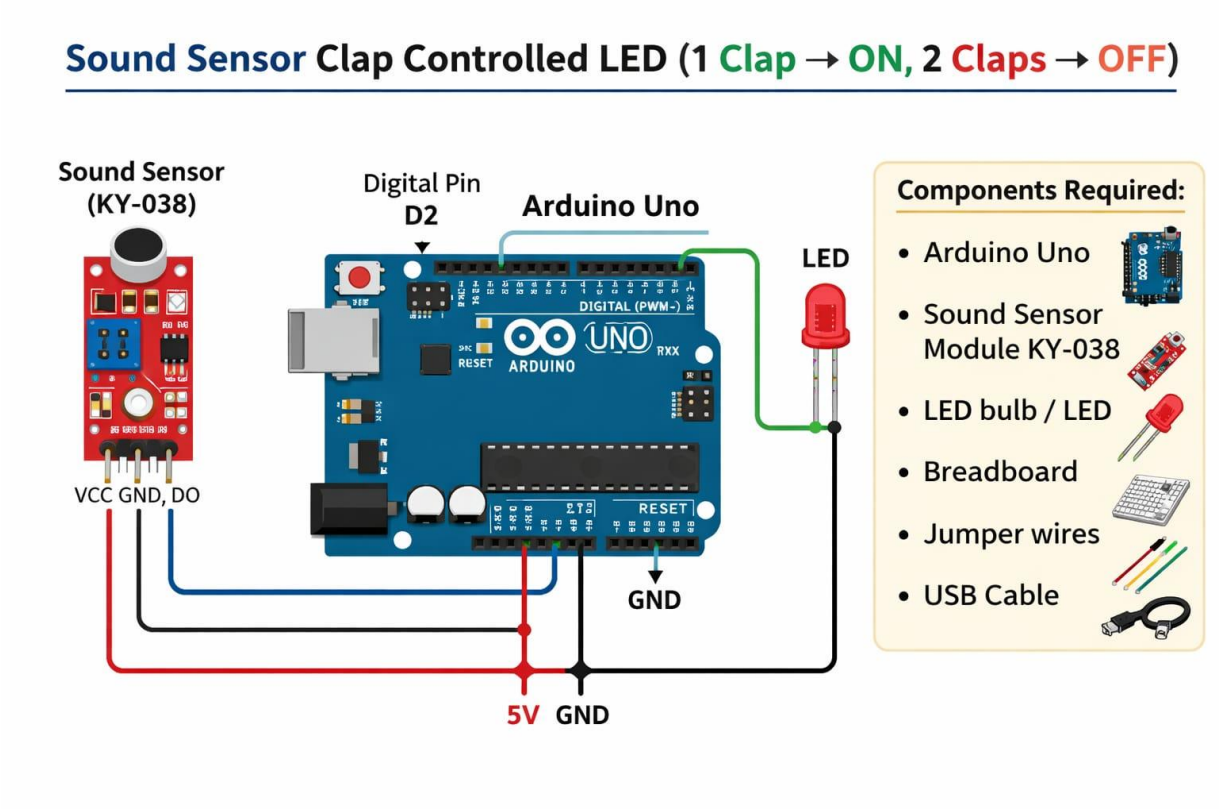
Working Principle:

1. The **sound sensor detects a clap sound**.
2. The sensor sends a signal to the **Arduino**.
3. Arduino counts the number of claps.
4. Based on the clap count:
    - **1 clap → LED bulb turns ON**
    - **2 claps → LED bulb turns OFF**

Applications:

- Smart home lighting systems
- Clap switch for lights and fans
- Assistive devices for elderly people
- Simple automation projects

Output:



RESULT:

The program has been executed successfully.

EX NO: 6	LDR Sensor with NodeMCU ESP8266
Date: 13.01.2026	

AIM:

To interface an LDR (Light Dependent Resistor) sensor with NodeMCU and control an LED based on light intensity.

Objective:

- Understand working of LDR sensor
- Learn analog input reading
- Control output based on sensor value
- Display readings in Serial Monitor

Components Required:

- NodeMCU ESP8266
- LDR sensor module
- LED
- Breadboard
- Jumper wires
- USB cable
- Arduino IDE

Theory:

An LDR sensor changes its resistance based on light intensity. Bright light gives low resistance and darkness gives high resistance. NodeMCU reads this as an analog signal and controls an LED.

Connections:

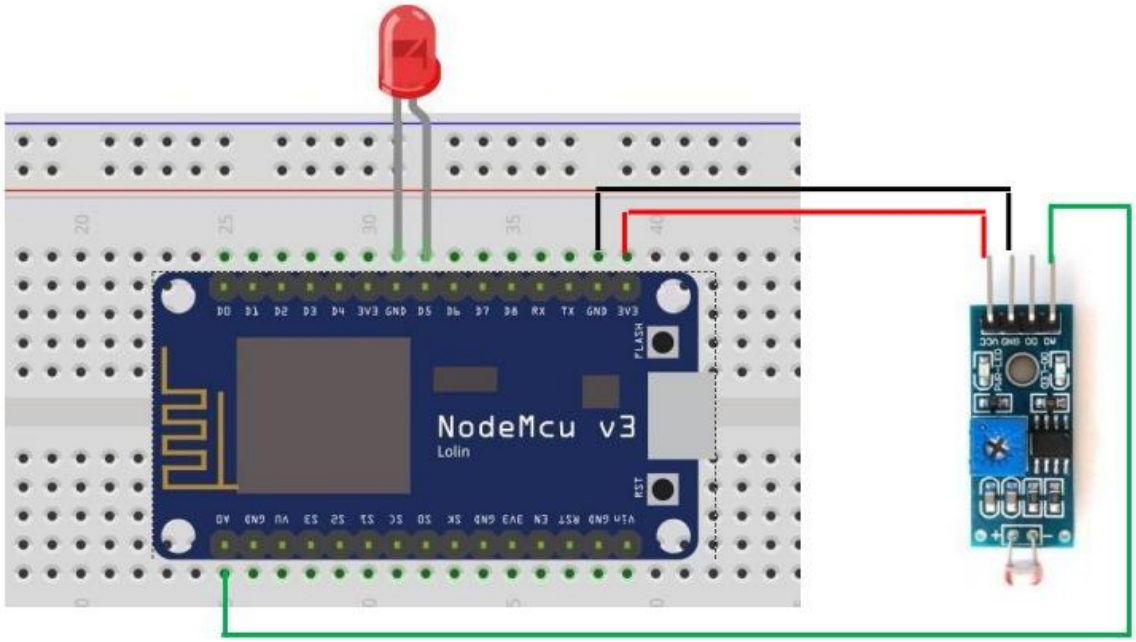
- LDR OUT → A0
- LDR VCC → 3.3V
- LDR GND → GND
- LED → D5

Program:

```
const int LDR_PIN = A0;

const int LED_PIN = D5;

void setup() {

  Serial.begin(9600);

  pinMode(LED_PIN, OUTPUT);

}
```

```
void loop() {
  int lightValue = analogRead(LDR_PIN);

  if (lightValue < 500) {
    digitalWrite(LED_PIN, HIGH);
  } else {
    digitalWrite(LED_PIN, LOW);
  }

  delay(500);
}
```

Output:



RESULT:

The program has been executed successfully

EX NO: 7 Date: 22.01.2026	LED Blinking Based on Soil Moisture using NodeMCU ESP8266

AIM:

To interface a soil moisture sensor with NodeMCU ESP8266 and make an LED blink when the soil becomes dry.

Apparatus Required:

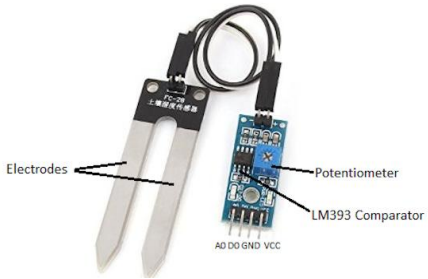
- NodeMCU ESP8266
- Soil Moisture Sensor (Probe + Control Module)
- LED
- 220 Ohm Resistor
- Breadboard
- Jumper Wires
- USB Cable
- Arduino IDE Software

Theory:

A soil moisture sensor detects the water content present in soil by measuring electrical conductivity. Wet soil conducts electricity better and produces lower analog values, while dry soil produces higher values. NodeMCU ESP8266 reads the analog value from the sensor through pin A0. In this experiment, a threshold value is used to determine whether the soil is dry or wet. If the soil becomes dry, the NodeMCU activates an LED in blinking mode, demonstrating basic automation.

Circuit Connections:

- Sensor VCC -> NodeMCU 3V3
- Sensor GND -> NodeMCU GND
- Sensor AO -> NodeMCU A0
- LED Cathode (-) -> GND



Program:

```
#define SOILPIN A0
#define LEDPIN D5
#define DRY_THRESHOLD 750

void setup() {
  Serial.begin(115200);
  pinMode(LEDPIN, OUTPUT);
}

void loop() {
  int moistureValue = analogRead(SOILPIN);

  Serial.print("Soil Moisture Value: ");
  Serial.println(moistureValue);

  if (moistureValue > DRY_THRESHOLD) {
    digitalWrite(LEDPIN, HIGH);
    delay(500);
    digitalWrite(LEDPIN, LOW);
    delay(500);
  } else {
    digitalWrite(LEDPIN, LOW);
  }
}
```

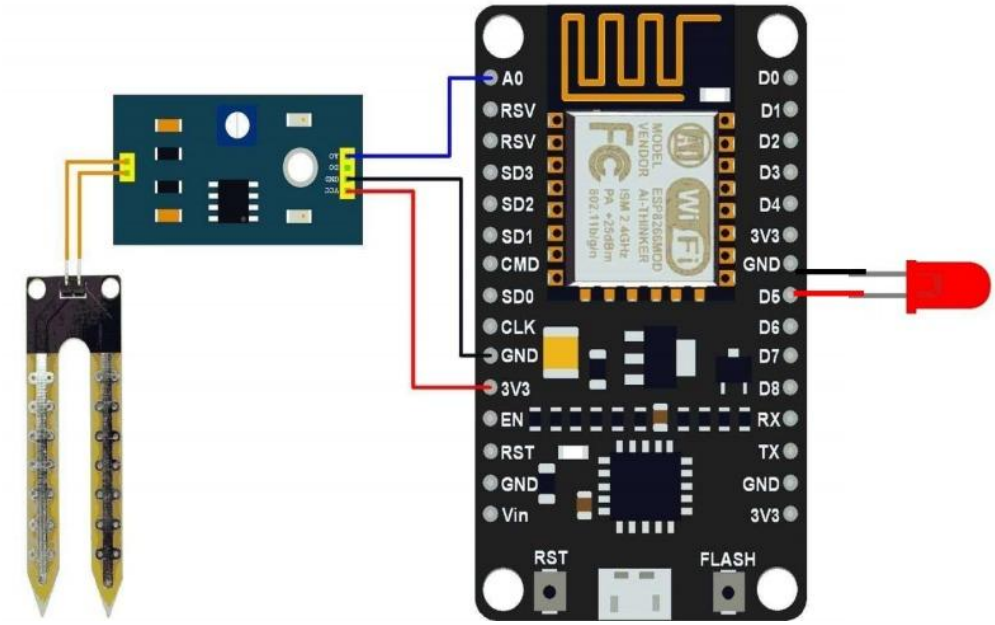
Procedure:

- Connect the soil moisture sensor and LED to NodeMCU as per the circuit connections.
- Open Arduino IDE and select NodeMCU 1.0 (ESP-12E Module).
- Choose the correct COM port.
- Upload the program to NodeMCU.
- Open Serial Monitor at 115200 baud rate.
- Insert the probe into soil and observe LED blinking when soil becomes dry.

Observation:

The LED blinks when the soil moisture value exceeds the predefined threshold, indicating dry soil condition.

Output:



RESULT:

The program has been executed successfully

EX NO: 8	Interfacing Temperature & Humidity Sensor with NodeMCU ESP8266
Date: 30.01.2026	

AIM:

To interface a DHT11/DHT22 temperature and humidity sensor with NodeMCU ESP8266 and display the environmental readings using Arduino IDE.

Apparatus Required:

- NodeMCU ESP8266
- DHT11 or DHT22 Sensor
- Breadboard
- Jumper Wires
- USB Cable
- Arduino IDE Software

Theory:

The DHT11/DHT22 is a digital sensor used to measure temperature and relative humidity. It consists of a capacitive humidity sensing element and a thermistor for temperature measurement. The internal microcontroller converts analog signals into calibrated digital output. NodeMCU ESP8266 reads this digital data through a GPIO pin and displays the values on the Serial Monitor. This experiment demonstrates embedded system interfacing and environmental monitoring.

Circuit Connections:

- DHT VCC -> NodeMCU 3V3
- DHT GND -> NodeMCU GND
- DHT DATA -> NodeMCU D4 (GPIO2)

Program:

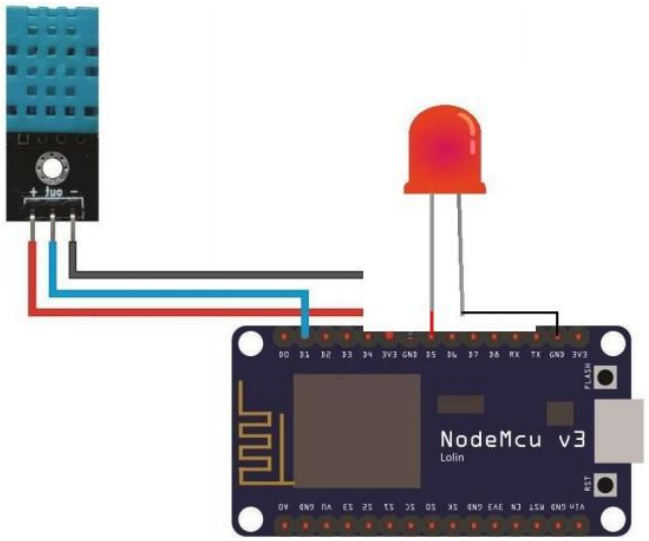
```
#include <ESP8266WiFi.h>
#include <DHT.h>

#define DHTPIN D4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
}

void loop() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();
```

```
  if (isnan(humidity) || isnan(temperature)) {
     Serial.println("Sensor error");
     return;
    }

    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.print(" °C  Humidity: ");
    Serial.print(humidity);
    Serial.println(" %");

    delay(2000);
  }
```

Procedure:

- Connect the DHT sensor to NodeMCU as per the circuit diagram.
- Install ESP8266 board package in Arduino IDE.
- Install DHT sensor library from Library Manager.
- Select NodeMCU 1.0 (ESP-12E Module) board.
- Upload the program to NodeMCU.
- Open Serial Monitor at 115200 baud rate.
- Observe temperature and humidity readings.

Output:



RESULT:

The program has been executed successfully.

EX NO: 9	LED Blinking Based on Temperature using NodeMCU ESP8266 and DHT Sensor
Date: 06.02.2026	

AIM:

To interface a DHT11/DHT22 temperature sensor with NodeMCU ESP8266 and blink an LED when the temperature falls below a predefined threshold value.

Apparatus Required:

- NodeMCU ESP8266
- DHT11 or DHT22 Sensor
- LED
- 220 Ohm Resistor
- Breadboard
- Jumper Wires
- USB Cable
- Arduino IDE Software

Theory:

The DHT sensor measures temperature and humidity and provides digital output. NodeMCU ESP8266 reads this data through a GPIO pin. In this experiment, the measured temperature is compared with a predefined threshold value. If the temperature is below the threshold, the NodeMCU activates an LED connected to a digital output pin. This demonstrates conditional control and basic automation using embedded systems.

Circuit Connections:

- DHT VCC -> NodeMCU 3V3
- DHT GND -> NodeMCU GND
- DHT DATA -> NodeMCU D4 (GPIO2)
- LED Anode (+) -> NodeMCU D5 (GPIO14) through 220 Ohm resistor
- LED Cathode (-) -> GND

Program:

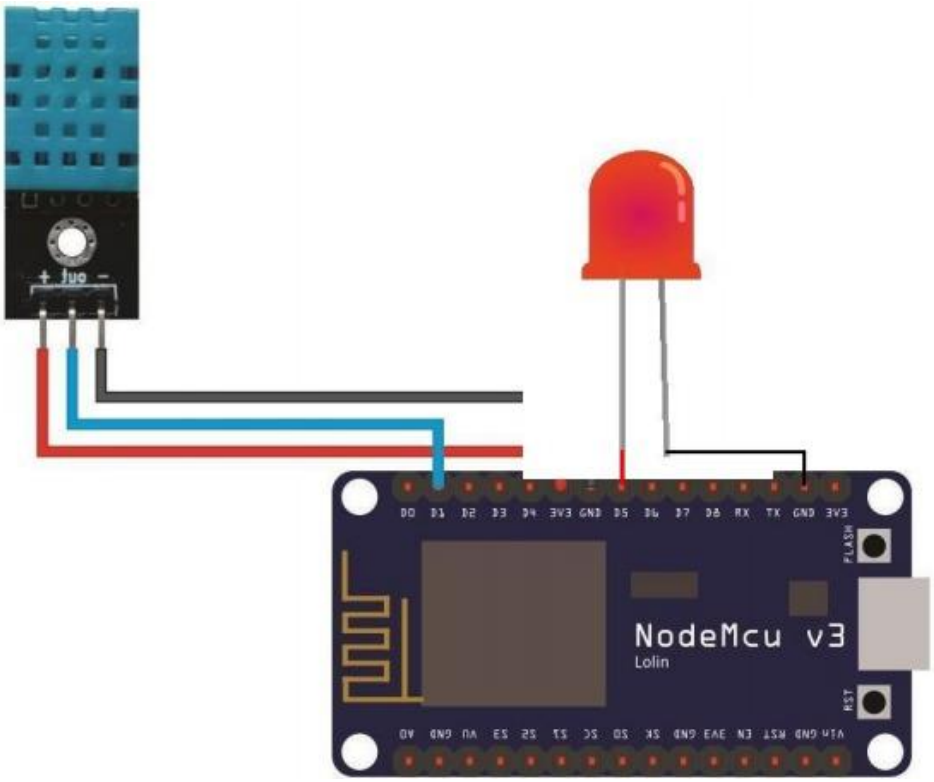
```
#include <ESP8266WiFi.h>
#include <DHT.h>

#define DHTPIN D4
#define DHTTYPE DHT11
#define LEDPIN D5
#define TEMP_LIMIT 25

DHT dht(DHTPIN, DHTTYPE);
```

```
void setup() {
  Serial.begin(115200);
  pinMode(LEDPIN, OUTPUT);
  digitalWrite(LEDPIN, LOW);
  dht.begin();
}

void loop() {
  float temperature = dht.readTemperature();

  if (isnan(temperature)) {
    Serial.println("Sensor error");
    return;
  }

  Serial.print("Temperature: ");
  Serial.println(temperature);

  if (temperature < TEMP_LIMIT) {
    digitalWrite(LEDPIN, HIGH);
  } else {
    digitalWrite(LEDPIN, LOW);
  }

  delay(2000);
}
```

Procedure:

- Connect the DHT sensor and LED to NodeMCU as per the circuit connections.
- Install ESP8266 board package in Arduino IDE.
- Install DHT sensor library from Library Manager.
- Select NodeMCU 1.0 (ESP-12E Module) board.
- Upload the program to NodeMCU.
- Open Serial Monitor at 115200 baud rate.
- Observe LED behavior when temperature changes.

Output:



RESULT:

The program has been executed successfully.

EX NO: 10	Weather Monitoring System using Django
Date: 13.02.2026	

AIM:

To develop a Weather Monitoring Web Application using Django that retrieves real-time weather information from the OpenWeatherMap API and displays temperature, humidity, and weather description.

Objective:

- Understand Django web framework
- Fetch data from an external API
- Display dynamic data using HTML templates

Software Requirements:

- Python 3.x
- Django
- Requests Library
- VS Code or any Python IDE
- Internet Connection

Theory:

Django is a Python-based web framework used for rapid web development. In this experiment, the system fetches weather data from the OpenWeatherMap API. The API returns data in JSON format which is processed by Django and displayed using an HTML template.

Commands Used:

- pip install django
- pip install requests
- python -m django startproject weather_project
- cd weather_project
- python manage.py startapp weather
- python manage.py runserver

Algorithm:

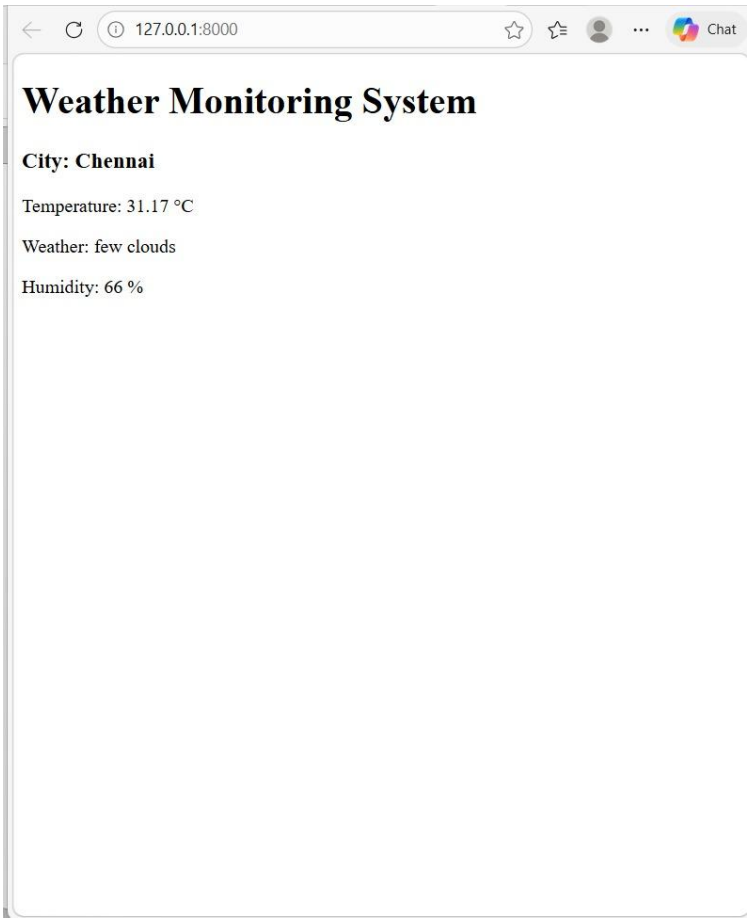
- Start the system
- Install required packages
- Create Django project
- Create Django app
- Fetch weather data using API

- Process JSON data
- Display results in HTML template
- Stop the system

Expected Output:

- Weather Monitoring System
- City: Chennai
- Temperature: 32 °C
- Weather: scattered clouds
- Humidity: 65 %

Output:



RESULT:

The program has been executed successfull

EX NO: 11 Date: 20.02.2026	Control an LED using Cloud BLYNK APP

AIM:

To design and implement a system that controls an LED remotely using the Blynk IoT App, enabling wireless operation through a smartphone via cloud connectivity.

Required Hardware :

1. Node MCU
2. LED
3. Bread Board

Control an LED using Cloud BLYNK APP

4. Micro USB Cable

Software :

1. Arduino IDE
2. Blynk App

1. Hardware Connection :

1. Connect LED +ve pin with D5 pin
2. Connect LED -ve pin with GND

2.Schematic :

Setting Up Blynk with NodeMCU (ESP8266)

1. Identify Components:

o NodeMCU board: This is the microcontroller that we'll use to connect to Blynk.
o LED bulb: We'll control this LED using Blynk.
o 180-ohm resistor: To limit the current through the LED.
o Breadboard: For connecting components.
o Jumper wires: To make connections.2. Connect Components:
o Follow the circuit diagram below to connect the components: Circuit Diagram
▪ Connect the NodeMCU's D1 (GPIO5) pin to the anode (longer leg) of the LED through the 180-ohm resistor.
▪ Connect the LED's cathode (shorter leg) to the NodeMCU's GND pin.
▪ Power the NodeMCU using a USB cable connected to your computer.

3. Set Up Blynk Web Application:

o Open your browser and go to the Blynk website.
o Click the "Start Free" button and sign up using your Gmail address.
o Check your registered email and click the new Blynk email.
o Create a strong password and include your profile name.
o Now you'll see the Blynk web app interface.
o Click the "Template" button and create your first template:
▪ Include a template name, device name, and connection type.
▪ Click "Done".

o Go to the “Datastreams” tab:

▪ Select the Virtual PIN and enter a name of your choice.
▪ Set the PIN as V0 and data type as integer.
▪ Click “Create”.
o Go to the “Web dashboard” tab:
▪ Drag and drop widgets (e.g., buttons) onto the dashboard.
▪ Configure the widgets (e.g., link them to the datastream we created earlier).
▪ Click “Save”.
o Create a new device:
▪ Click the “Search” icon and select your template.
▪ The Blynk web app is now ready.

4. Set Up Blynk Mobile App:

o Download and install the Blynk app from the Play Store or App Store on your phone.
o Sign in with the email and password you created in the web application.o You’ll see the template you created in the web app.
o Customize your mobile dashboard by adding widgets (e.g., buttons).
o Link the widgets to the datastream.
Arduino Script
Now write following code and upload it into the Board using upload button

```
#define BLYNK_TEMPLATE_ID "TMPL3xQMixHTe"
#define BLYNK_TEMPLATE_NAME "WaterLevel"
#define BLYNK_AUTH_TOKEN "7Wv9jrAq07ykWD7dFr2wNF8q-KMvHZo8"
#define BLYNK_PRINT Serial
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
// WiFi credentials
char ssid[] = "xxxxxxxxx";
char pass[] = "xxxxxxxxx";
BlynkTimer timer;
BLYNK_WRITE(V0) {
int value = param.asInt();
Serial.println(value);
digitalWrite(D5, value);
}BLYNK_CONNECTED() {
Blynk.virtualWrite(V1, "Connected");
}
void setup() {
Serial.begin(115200);
Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
pinMode(D5, OUTPUT); // Set D5 as output
timer.setInterval(1000L, []() {
Blynk.virtualWrite(V2, millis() / 1000);
});
}
void loop() {
Blynk.run();
timer.run();
}
```

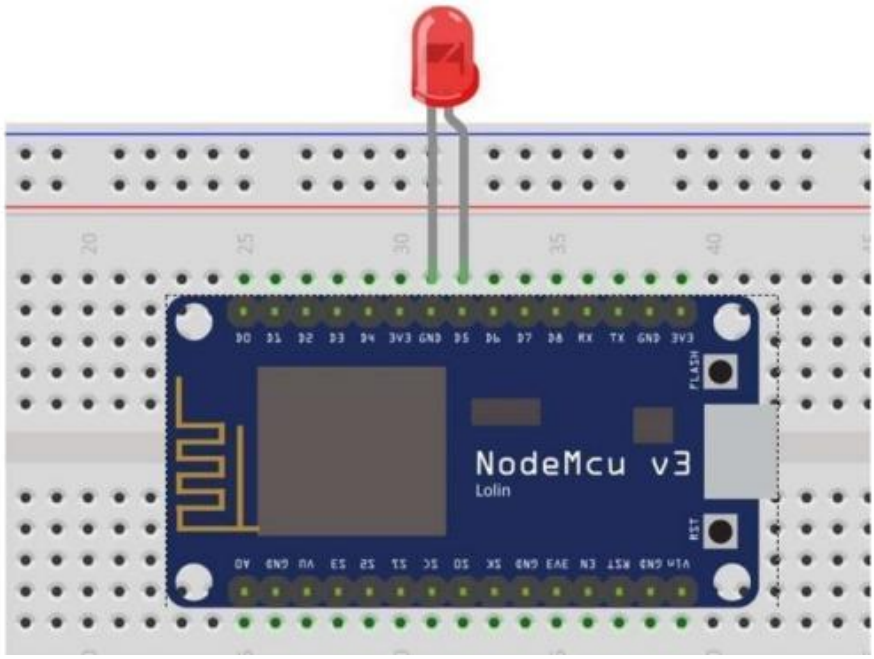
Step 6: Select Tools → Board → NodeMCU 1.0 (ESP- 12E Module).

Step 7: Install Blynk library from Sketch → Include Library → Manage Libraries.
Step 8: Connect NodeMCU using USB cable and select the COM port.
Step 9: Write and upload the program.

Program

```
#define BLYNK_PRINT Serial
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

char auth[] = "Your_Blynk_Token";
char ssid[] = "Your_WiFi_Name";
char pass[] = "Your_WiFi_Password";

void setup()
{
  Serial.begin(9600);
  Blynk.begin(auth, ssid, pass);
}

void loop()
{
  Blynk.run();
```

Output:



RESULT:

The program has been executed successfully.

EX NO: 12	Gas Detection Using MQ-2 Gas Sensor with ESP8266 (NodeMCU)
Date: 27.02.2026	

AIM:

To design and implement a gas detection system using the MQ-2 Gas Sensor and ESP8266 NodeMCU to monitor the presence of combustible gases and provide alerts in real-time for safety applications

Required Hardware :

1. Node MCU ESP 8266
2. MQ4 Gas Sensor
3. Bread Board
4. Micro USB Cable
5. LED or Buzzer

Software :

1. Arduino IDE

Hardware Connection :

1. Connect MQ4 VCC pin with 3.3v pin in Node MCU
2. Connect MQ4 GND pin with GND in Node MCU
3. Connect MQ4 A0 pin with A0 in Node MCU
4. Connect Buzzer +ve pin with D2
5. Connect Buzzer -ve pin with GND

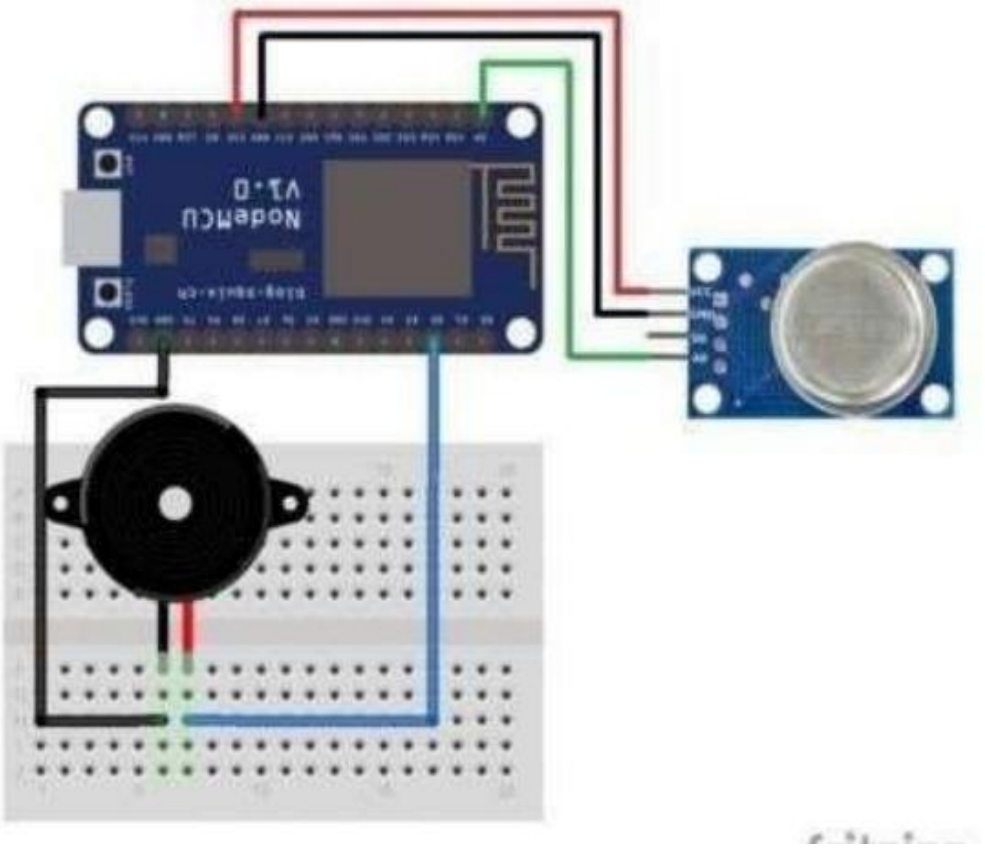
Program:

Now write following code and upload it into the Board using upload button

```
#include <ESP8266WiFi.h>
#define smokeA0 A0 // Smoke sensor connected to A0
#define Alarm D2 // Alarm connected to D2
int sensorThres = 500; // Threshold value for smoke detection
void setup() {
pinMode(Alarm, OUTPUT); // Set Alarm as output
pinMode(smokeA0, INPUT); // Set Smoke Sensor as input
Serial.begin(9600); // Start serial communication
}
void loop() {
int sensorValue = analogRead(smokeA0); // Read smoke sensor value
Serial.print("Sensor Value: ");
Serial.println(sensorValue); // Print sensor value
if (sensorValue > sensorThres) {
digitalWrite(Alarm, HIGH); // Turn ON alarm
```

```
    } else {
    digitalWrite(Alarm, LOW); // Turn OFF alarm
    }
delay(100); // Short delay for stability
}
```

Output:



RESULT:

The program has been executed successfully.

EX NO: 13	IoT-Based Object Detection System using ESP8266 and Django
Date: 27.02.2026	

Aim:

To develop an IoT system that detects objects using an IR sensor and sends data to a Django web server.

Objective:

- Interface IR sensor with ESP8266
- Send data using HTTP POST
- Receive data in Django server

Hardware Requirements:

ESP8266 NodeMCU
IR Sensor
Breadboard
Jumper wires
WiFi-enabled PC

Software Requirements:

Python
Django
Arduino IDE
Requests Library

Theory:

IoT connects devices to the internet. The IR sensor detects objects, ESP8266 sends data, and Django receives it.

Circuit Connection:

IR VCC → 3.3V
IR GND → GND
IR OUT → D5

Algorithm:

1. Start system
2. Connect ESP8266 to WiFi
3. Read IR sensor
4. If object detected → "Detected"
5. Else → "No Object"
6. Send data

7. Repeat

Django Setup (Python):

```
pip install django
django-admin startproject iot_project
cd iot_project
python manage.py startapp sensor
```

Django View Code:

```
from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt

@csrf_exempt
def sensor_data(request):
    if request.method == 'POST':
        status = request.POST.get('status')
        print("Sensor Status:", status)
        return JsonResponse({'message': 'Data received'})
    return JsonResponse({'error': 'Invalid request'})
```

URL Configuration:

```
from django.urls import path
from .views import sensor_data

urlpatterns = [
    path('api/', sensor_data),
]
```

Run Server:

```
python manage.py runserver 0.0.0.0:8000
```

ESP8266 Code:

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
const char* serverName = "http://YOUR_PC_IP:8000/api/";

int IR_PIN = D5;

void setup() {
  Serial.begin(115200);
  pinMode(IR_PIN, INPUT);
  WiFi.begin(ssid, password);
```

```
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting...");

  }
}

void loop() {
  if (WiFi.status() == WL_CONNECTED) {
    WiFiClient client;
    HTTPClient http;

    int sensorValue = digitalRead(IR_PIN);
    String status;

    if (sensorValue == LOW) {
      status = "Object Detected";
    } else {
      status = "No Object";
    }

    http.begin(client, serverName);
    http.addHeader("Content-Type", "application/x-www-form-urlencoded");

    String httpRequestData = "status=" + status;
    http.POST(httpRequestData);

    Serial.println(status);
    http.end();
  }

  delay(2000);
}
```

Expected Output:

Sensor Status: Object Detected
Sensor Status: No Object

RESULT:

The program has been executed successfully.

EX NO: 14	LDR Based - IoT Monitoring system
Date: 07.03.2026	

Aim:

To measure light intensity using LDR and send data to a Django server.

Procedure:

Step 1: Setup Hardware
- Connect LDR to 3.3V
- Connect resistor to GND
- Connect junction to A0

Step 2: Install Software
pip install django

Step 3: Create Django Project
django-admin startproject iot_project
cd iot_project
python manage.py startapp sensor

Step 4: Django Code (views.py)

```
from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt

@csrf_exempt
def ldr_data(request):
    if request.method == 'POST':
        status = request.POST.get('status')
        print("LDR Status:", status)
        return JsonResponse({'msg': 'received'})
```

Step 5: URL Configuration

```
from django.urls import path
from .views import ldr_data

urlpatterns = [
    path('ldr/', ldr_data),
]
```

Step 6: Run Server
python manage.py runserver 0.0.0.0:8000

Step 7: ESP8266 Code

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

const char* ssid = "YOUR_WIFI";
```

```
const char* password = "YOUR_PASSWORD";
const char* serverName = "http://YOUR_PC_IP:8000/ldr/";

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
  }
}

void loop() {
  if (WiFi.status() == WL_CONNECTED) {
    WiFiClient client;
    HTTPClient http;

    int ldrValue = analogRead(A0);
    String status;

    if (ldrValue < 500) status = "Dark";
    else status = "Bright";

    http.begin(client, serverName);
    http.addHeader("Content-Type", "application/x-www-form-urlencoded");
    http.POST("status=" + status);

    Serial.println(status);
    http.end();
  }
  delay(2000);
}
```

Step 8: Upload Code and Test
- Upload code
- Cover LDR → Dark
- Light → Bright

RESULT:
The program has been executed successfully.

EX NO: 15	IR Sensor Based People Counter (IoT Lab Manual)
Date: 13.03.2026	

Aim:

To count the number of people passing through using an IR sensor and display the count using ESP8266.

Objective:

- Detect object using IR sensor
- Count each detection
- Display count in Serial Monitor

Components Required:

ESP8266 NodeMCU
IR Sensor
Jumper wires
Breadboard

Circuit Connection:

IR VCC → 3.3V
IR GND → GND
IR OUT → D5

Theory:

The IR sensor detects objects. Each detection increments a counter in ESP8266. The count is displayed in the Serial Monitor.

Algorithm:

1. Initialize count = 0
2. Read IR sensor
3. If object detected, increase count
4. Wait until object leaves
5. Repeat

ESP8266 Code:

```
int IR_PIN = D5;
int count = 0;
bool objectDetected = false;

void setup() {
  Serial.begin(115200);
  pinMode(IR_PIN, INPUT);
}
```

```
void loop() {
  int sensorValue = digitalRead(IR_PIN);

  if (sensorValue == LOW && objectDetected == false) {
    count++;
    objectDetected = true;

    Serial.print("People Count: ");
    Serial.println(count);

    delay(500);
  }

  if (sensorValue == HIGH) {
    objectDetected = false;
  }
}
```

Procedure:

Step 1: Connect IR sensor to ESP8266
Step 2: Open Arduino IDE
Step 3: Select NodeMCU board
Step 4: Paste and upload code
Step 5: Open Serial Monitor (115200 baud)
Step 6: Move object in front of sensor

Observation Table:

S.No	Trial	IR Status	Action	Count
1	1	LOW	Detected	1
2	2	LOW	Detected	2
3	3	LOW	Detected	3

Expected Output:

People Count: 1
People Count: 2
People Count: 3

RESULT:
The program has been executed successfully.